

Module 2 – Introduction to Programming Overview of C Programming

• THEORY EXERCISE:

Q1. Write an essay covering the history and evolution of C programming. Explain its importance and why it is still used today.

The C programming language stands as a cornerstone of modern computing, its influence pervasive across a vast spectrum of software and hardware. Its history is a testament to the pursuit of efficiency and portability, and its continued relevance underscores its enduring power.

Here's an exploration of C's history, evolution, and enduring importance:

Historical Roots and Evolution:

- . Origins at Bell Labs:**

- **The C language was developed at Bell Laboratories in the early 1970s by Dennis Ritchie.**
- **It evolved from earlier languages, notably BCPL (Basic Combined Programming Language) and B, developed by Ken Thompson.**
- **C was initially created to develop the Unix operating system.**
- **Key Developments:**
 - **The language's design prioritized efficiency, flexibility, and portability, allowing it to run on various hardware platforms.**
 - **The publication of "The C Programming Language" by Brian Kernighan and Dennis Ritchie (often referred to as K&R) in 1978 significantly popularized the language.**
 - **Standardization:**

- **The need for a standardized version of C led to the development of ANSI C (C89/C90) in 1989, providing a consistent definition of the language.**
- **Subsequent standards, such as C99, C11, and C18, have introduced further enhancements and refinements.**
- **Evolution:**
 - **C has been continually updated with new standardizations, to keep up with changing computing needs. These updates have added features like, variable length arrays, multi threading support, and many other improvements.**

Importance and Enduring Relevance:

- **Foundation for Other Languages:**
 - **C has served as a foundational language for many other programming**

languages, including C++, Java, and Python.

- **Understanding C provides a deep understanding of core programming concepts.**

- **System Programming:**

- **C remains the language of choice for system programming, including operating system kernels, device drivers, and embedded systems.**
- **Its low-level access to hardware and efficient memory management make it ideal for these applications.**

- **Performance and Efficiency:**

- **C's ability to produce highly optimized code makes it suitable for performance-critical applications.**
- **It's widely used in areas such as game development, high-frequency trading, and scientific computing.**

- **Embedded Systems:**
 - **C's small footprint and hardware access capabilities make it essential for embedded systems, which are found in a wide range of devices, from automobiles to household appliances.**
- **Legacy Systems:**
 - **Many legacy systems and applications are written in C, requiring ongoing maintenance and development.**

Why C Is Still Used Today:

- **Speed and Efficiency:**
 - **C's close-to-hardware nature allows for highly optimized code, making it exceptionally fast.**
- **Portability:**

- **C code can be compiled and run on a wide variety of platforms, contributing to its versatility.**
- **Control:**
 - **C provides a high degree of control over memory management and hardware access, which is crucial for certain applications.**
- **Established Ecosystem:**
 - **A vast ecosystem of libraries and tools supports C development, making it a robust and reliable choice.**

• **LAB EXERCISE:**

Q1. Research and provide three real-world applications where C programming is extensively used, such as in embedded systems, operating systems, or game development.

It's true that C remains a very relevant and widely used language in many critical

applications. Here are three real-world areas where C programming is extensively used:

- . Operating Systems:**

- **C's efficiency and low-level access to hardware make it the ideal choice for developing operating system kernels.**
- **The Linux kernel, a cornerstone of many modern operating systems, is written in C.**
- **Similarly, parts of Microsoft Windows are also written in C.**
- **C's ability to directly interact with hardware resources is essential for managing system processes, memory, and devices.**

- . Embedded Systems:**

- **Embedded systems are found in a vast array of devices, from microcontrollers in household**

appliances to complex systems in automobiles and aircraft.

- **C's small footprint and ability to produce highly efficient code make it perfect for these resource-constrained environments.**
 - **Device drivers, which enable communication between hardware and software, are often written in C.**
 - **Example: automotive control systems, medical devices, and industrial automation.**
- . Database Management Systems:**
- **Database management systems (DBMS) require high performance and efficient memory management.**
 - **C's capabilities make it well-suited for developing the core components of DBMS software.**

- **Examples: MySQL and PostgreSQL, which are popular open-source database systems, are written in C.**
- **C allows for the needed speed and stability for those kinds of systems.**

2. Setting Up Environment

• THEORY EXERCISE:

Q1. Describe the steps to install a C compiler (e.g., GCC) and set up an Integrated Development Environment (IDE) like DevC++, VS Code, or CodeBlocks.

Let's break down the installation of GCC (GNU Compiler Collection) and the setup of a few popular IDEs for C programming.

1. Installing GCC (GNU Compiler Collection):

GCC is the standard C compiler on Linux and macOS. On Windows, you'll typically use MinGW or WSL.

- **Linux (Debian/Ubuntu based):**
 - **Open a terminal.**

- **Update the package list: \$ sudo apt update**
- **Install GCC and build-essential (includes necessary tools): \$ sudo apt install build-essential**
- **Verify the installation: \$ gcc --version**
- **macOS:**
 - **If you don't have Xcode Command Line Tools, open a terminal and run: \$ xcode-select --install**
 - **This will install GCC (clang, which is compatible and often used as a replacement for GCC on macOS).**
 - **Verify the installation: \$ gcc --version**
- **Windows (MinGW):**
 - **Download the MinGW installer from a reliable source (e.g., SourceForge).**
 - **Run the installer.**

- **Select the "gcc-core" and "g++" packages (for C and C++). You may also want "mingw32-make"**
- **Add the MinGW "bin" directory to your system's PATH environment variable.**
 - **This allows you to run GCC from the command prompt.**
- **Verify the installation: open a command prompt and type \$ gcc --version**
- **Windows (WSL - Windows Subsystem for Linux):**
 - **Enable WSL on your Windows machine.**
 - **Install a linux distribution from the Microsoft store (Ubuntu, Debian, etc.)**
 - **Open the linux terminal that you have installed.**
 - **Follow the Linux installation instructions above.**

2. Setting up an Integrated Development Environment (IDE):

Here's how to set up three popular IDEs:

- . Dev-C++:**

- **Download Dev-C++ from a reliable source.**
- **Run the installer.**
- **Dev-C++ typically comes with a bundled GCC compiler, so you might not need to install it separately.**
- **To create a new project:**
 - . File -> New -> Project -> Console Application -> C Project.**
 - . Write your C code in the editor.**
 - . Compile and run your code using the "Execute" menu.**

- . VS Code (Visual Studio Code):**

- **Download and install VS Code from the official website.**

- **Install the "C/C++" extension from Microsoft.**
- **Install GCC (as described above).**
- **Create a new folder for your project.**
- **Open the folder in VS Code.**
- **Create a '.c' file within the folder.**
- **To compile and run from the terminal inside VS code:**
 - **Open the integrated terminal (View -> Terminal).**
 - **Compile: `$ gcc your_file.c -o your_program`**
 - **Run: `$ ./your_program`**
- **You can also set up VS Code tasks for automated builds and debugging.**
- **Code::Blocks:**
 - **Download Code::Blocks from the official website.**
 - **Run the installer.**

- **Code::Blocks usually detects installed GCC compilers.**
- **To create a new project:**
 - **File -> New -> Project -> Console application -> Go -> C -> next.**
 - **Write your C code in the editor.**
 - **Build and run your code using the build and run buttons.**
- **It offers a user friendly GUI, and project management tools.**

• LAB EXERCISE:

Q1. Install a C compiler on your system and configure the IDE. Write your first program to print "Hello, World!" and run it.

1. Installing GCC (on Windows, using MinGW):

- **Download MinGW:**

- **Go to SourceForge and download the MinGW installer (e.g., mingw-get-setup.exe).**
- **Run the Installer:**
 - **Execute the installer.**
 - **Choose the installation directory (e.g., C:\MinGW).**
 - **In the package selection window, select mingw32-base, mingw32-gcc-g++, and mingw32-make.**
 - **Click "Installation" -> "Apply Changes."**
- **Add MinGW to PATH:**
 - **Search for "environment variables" in the Windows search bar and select "Edit the system environment variables."**
 - **Click "Environment Variables."**
 - **In the "System variables" section, find the "Path" variable and click "Edit."**

- Click "New" and add the path to the MinGW "bin" directory (e.g., C:\MinGW\bin).
- Click "OK" on all open windows.
- **Verify Installation:**
 - Open a command prompt (cmd).
 - Type `gcc --version` and press Enter. You should see the GCC version information.

2. Installing and Configuring VS Code:

- **Install VS Code:**
 - Download and install VS Code from the official website.
- **Install the C/C++ Extension:**
 - Open VS Code.
 - Click on the Extensions icon (square icon on the left sidebar).
 - Search for "C/C++" and install the extension from Microsoft.

- **Create a Project Folder:**

- **Create a new folder on your computer (e.g., C_Projects).**
- **Open VS Code and then open the folder you just created through the file menu.**

3. Writing the "Hello, World!" Program:

- **Create a C File:**

- **In VS Code, create a new file named hello.c in the project folder.**

- **Write the Code:**

- **Open hello.c and enter the following code:**

C

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Hello, World!\n");
```

```
    return 0;
```

}

- **Save the File:**
 - **Save the file (Ctrl + S).**

4. Compiling and Running the Program:

- **Open the Terminal:**
 - **In VS Code, open the integrated terminal (View -> Terminal).**
- **Compile the Code:**
 - **In the terminal, type the following command and press Enter:**

Bash

gcc hello.c -o hello

- **This command compiles the hello.c file and creates an executable file named hello.exe.**
- **Run the Executable:**
 - **In the terminal, type the following command and press Enter:**

Bash

./hello.exe

- . You should see "Hello, World!" printed in the terminal.**

3. Basic Structure of a C Program

- THEORY EXERCISE:**

Q1.Explain the basic structure of a C program, including headers, main function, comments, data types, and variables.

Provide examples.

Let's break down the basic structure of a C program, covering the essential components: headers, the main function, comments, data types, and variables.

1. Basic Structure:

A typical C program follows this general structure:

C

#include <header_file.h> // Header inclusion

// Global variables (optional)

```
int main() {  
    // Variable declarations  
    // Statements (code)  
    return 0; // Exit status  
}
```

// Function definitions (optional)

2. Headers:

- . Headers are files that contain declarations of functions, variables, and constants.**
- . They are included using the #include directive.**
- . Example:**
 - #include <stdio.h>: Includes the standard input/output library.**

- **#include <stdlib.h>: Includes the standard library.**
- **#include <math.h>: Includes ¹ mathematical functions.**

[1. www.chegg.com](http://www.chegg.com)

www.chegg.com

3. The main Function:

- **The main function is the entry point of every C program.**
- **It's where program execution begins.**
- **It returns an integer value, typically 0, to indicate successful execution.**
- **Example:**

C

```
int main() {  
    // Code goes here  
    return 0;  
}
```

4. Comments:

- Comments are used to explain code and improve readability.**
- C supports two types of comments:**
 - Single-line comments: // This is a single-line comment**
 - Multi-line comments: /* This is a multi-line comment */**

5. Data Types:

- Data types specify the type of data a variable can hold.**
- Basic data types include:**
 - int: Integer (e.g., -10, 0, 100)**
 - float: Floating-point number (e.g., 3.14, -2.5)**

- **double: Double-precision floating-point number (more precise than float)**
 - **char: Character (e.g., 'a', 'Z', '5')**
 - **void: Represents the absence of a type.**
- **Example:**

C

int age = 30;

float pi = 3.14159;

char grade = 'A';

6. Variables:

- **Variables are named storage locations that hold data.**
- **They must be declared with a data type before they can be used.**
- **Variable names must follow certain rules (e.g., start with a letter or underscore, no spaces).**

. Example:

C

int count; // Declaration

count = 10; // Assignment

int sum = 0; // Declaration and initialization

Example Program:

C

#include <stdio.h>

int main() {

// Variable declarations

int num1 = 10;

int num2 = 20;

int sum;

// Calculate the sum

sum = num1 + num2;


```
// Print the result  
  
printf("The sum of %d and %d is %d\n",  
num1, num2, sum);  
  
return 0; // Successful execution  
  
}
```

- **LAB EXERCISE:**

Q1. Write a C program that includes variables, constants, and comments. Declare and use different data types (int, char, float) and display their values.

```
#include <stdio.h> int main() { // Variable  
declarations int age = 30; // Integer variable  
for age char grade = 'A'; // Character variable  
for grade float pi = 3.14159f; // Float variable  
for pi (note the 'f' suffix) // Constant  
declaration (using const keyword) const int  
DAYS_IN_WEEK = 7; // Displaying variable  
values using printf printf("Age: %d\n", age);
```

```
printf("Grade: %c\n", grade); printf("Pi: %f\n",
pi); // Displaying constant value printf("Days
in a week: %d\n", DAYS_IN_WEEK); // Some
simple arithmetic operations int doubleAge =
age * 2; float piPlusOne = pi + 1.0f;
//Displaying results of arithmetic operations.
printf("Double age: %d\n", doubleAge);
printf("Pi + 1: %f\n", piPlusOne); //
Demonstrating character arithmetic char
nextGrade = grade + 1; // 'A' + 1 = 'B'
printf("Next Grade: %c\n", nextGrade); return
0; // Successful execution }
```

4. Operators in C

• THEORY EXERCISE:

Q1. Write notes explaining each type of operator in C: arithmetic, relational, logical, assignment, increment/decrement, bitwise, and conditional operators.

1. Arithmetic Operators:

- These operators perform mathematical calculations.
- Operators:
 - **+: Addition**
 - **-: Subtraction**
 - ***: Multiplication**
 - **/: Division**
 - **?: Modulus (remainder)**
- Example:

C

```
int a = 10, b = 3;
```

```
int sum = a + b;    // sum = 13
```

```
int difference = a - b; // difference = 7
```

```
int product = a * b;  // product = 30
```

```
int quotient = a / b; // quotient = 3
```

```
int remainder = a % b; // remainder = 1
```

2. Relational Operators:

- These operators compare two values and return a boolean result (true or false, represented as 1 or 0 in C).
- Operators:
 - ==: Equal to
 - !=: Not equal to
 - >: Greater than
 - <: Less than
 - >=: Greater than or equal to
 - <=: Less than or equal to
- Example:

C

```
int x = 5, y = 10;
```

```
int result1 = (x == y); // result1 = 0 (false)
```

```
int result2 = (x < y); // result2 = 1 (true)
```

```
int result3 = (x >= 5); // result3 = 1 (true)
```

3. Logical Operators:

- These operators combine or negate boolean expressions.
- Operators:
 - **&&**: Logical AND (true if both operands are true)
 - **||**: Logical OR (true if at least one operand is true)
 - **!**: Logical NOT (negates the operand)
- Example:

C

```
int a = 1, b = 0;
```

```
int result1 = (a && b); // result1 = 0 (false)
```

```
int result2 = (a || b); // result2 = 1 (true)
```

```
int result3 = !a;    // result3 = 0 (false)
```

4. Assignment Operators:

- These operators assign values to variables.
- Operators:

- **=:** Simple assignment
- **+=:** Add and assign
- **-=:** Subtract and assign
- ***=:** Multiply and assign
- **/=:** Divide and assign
- **%=:** Modulus and assign ¹

[1. shitanshumanhas.blogspot.com](http://1.shitanshumanhas.blogspot.com)

shitanshumanhas.blogspot.com

. Example:

C

```
int x = 10;
```

```
x += 5; // x = x + 5; x becomes 15
```

```
x *= 2; // x = x * 2; x becomes 30
```

5. Increment/Decrement Operators:

- These operators increase or decrease the value of a variable by 1.
- Operators:
 - ++: Increment
 - --: Decrement
- Prefix vs. Postfix:
 - ++x (prefix): Increments x before using its value.
 - x++ (postfix): Increments x after using its value.
- Example:

C

```
int a = 5;  
  
int b = ++a; // a = 6, b = 6  
  
int c = a++; // c = 6, a = 7
```

6. Bitwise Operators:

- These operators perform operations on individual bits of integer values.

- **Operators:**

- **&: Bitwise AND**
- **|: Bitwise OR**
- **^: Bitwise XOR (exclusive OR)**
- **~: Bitwise NOT (complement)**
- **<<: Left shift**
- **>>: Right shift**

- **Example:**

C

unsigned int a = 60; // 0011 1100

unsigned int b = 13; // 0000 1101

unsigned int result1 = a & b; // 0000 1100

(12)

unsigned int result2 = a << 2; // 1111 0000

(240)

7. Conditional (Ternary) Operator:

- **This operator provides a shorthand for if-else statements.**

- **Syntax: condition ? expression1 : expression2**
- **If condition is true, expression1 is evaluated; otherwise, expression2 is evaluated.**
- **Example:**

C

```
int age = 20;
```

```
char *message = (age >= 18) ? "Adult" :  
"Minor"; // message = "Adult"
```

• LAB EXERCISE:

Q1. Write a C program that accepts two integers from the user and performs arithmetic, relational, and logical operations on them. Display the results.

```
#include <stdio.h> int main() { int num1,  
num2; // Get input from the user printf("Enter  
the first integer: "); scanf("%d", &num1);  
printf("Enter the second integer: ");  
scanf("%d", &num2); // Arithmetic
```

```
Operations printf("\nArithmetic
Operations:\n"); printf("%d + %d = %d\n",
num1, num2, num1 + num2); printf("%d - %d
= %d\n", num1, num2, num1 - num2);
printf("%d * %d = %d\n", num1, num2, num1 *
num2); if (num2 != 0) { printf("%d / %d =
%d\n", num1, num2, num1 / num2);
printf("%d %% %d = %d\n", num1, num2,
num1 % num2); } else { printf("Division by
zero is not allowed.\n"); } // Relational
Operations printf("\nRelational
Operations:\n"); printf("%d == %d is %d\n",
num1, num2, num1 == num2); printf("%d !=
%d is %d\n", num1, num2, num1 != num2);
printf("%d > %d is %d\n", num1, num2, num1
> num2); printf("%d < %d is %d\n", num1,
num2, num1 < num2); printf("%d >= %d is
%d\n", num1, num2, num1 >= num2);
printf("%d <= %d is %d\n", num1, num2,
num1 <= num2); // Logical Operations
printf("\nLogical Operations:\n"); printf("(%d
> 0) && (%d < 10) is %d\n", num1, num2,
```

```
(num1 > 0) && (num2 < 10)); printf("(%d > 0) ||  
(%d < 10) is %d\n", num1, num2, (num1 > 0) ||  
(num2 < 10)); printf("!(%d == %d) is %d\n",  
num1, num2, !(num1 == num2)); return 0; }
```

5. Control Flow Statements in C

• THEORY EXERCISE:

Q1.Explain decision-making statements in C (if, else, nested if-else, switch). Provide examples of each.

Let's delve into decision-making statements in C, which allow your programs to execute different code blocks based on conditions.

1. if Statement:

- . The if statement executes a block of code if a specified condition is true.**
- . Syntax:**

C

```
if (condition) {  
    // Code to execute if condition is true
```

```
}
```

- **Example:**

C

```
int age = 20;  
if (age >= 18) {  
    printf("You are an adult.\n");  
}
```

2. else Statement:

- The else statement provides an alternative block of code to execute if the if condition is false.

- **Syntax:**

C

```
if (condition) {  
    // Code to execute if condition is true  
} else {  
    // Code to execute if condition is false  
}
```

- **Example:**

C

```
int age = 15;

if (age >= 18) {
    printf("You are an adult.\n");
} else {
    printf("You are a minor.\n");
}
```

3. nested if-else Statements:

- **Nested if-else statements allow you to check multiple conditions within other conditions.**
- **Syntax:**

C

```
if (condition1) {
    // Code to execute if condition1 is true
    if (condition2) {
```

```
// Code to execute if condition1 and  
condition2 are true  
  
} else {  
  
    // Code to execute if condition1 is true  
    and condition2 is false  
  
}  
  
} else {  
  
    // Code to execute if condition1 is false  
  
}
```

. Example:

C

```
int score = 85;  
  
if (score >= 90) {  
    printf("Grade: A\n");  
} else if (score >= 80) {  
    printf("Grade: B\n");  
} else if (score >= 70) {  
    printf("Grade: C\n");
```

```
} else {  
    printf("Grade: D or F\n");  
}
```

4. switch Statement:

- . The switch statement provides a more efficient way to handle multiple if-else conditions based on the value of a single variable.**
- . Syntax:**

C

```
switch (expression) {  
    case constant1:  
        // Code to execute if expression ==  
constant1  
        break;  
    case constant2:  
        // Code to execute if expression ==  
constant2  
        break;
```

```
// ... more cases  
default:  
    // Code to execute if no case matches  
}
```

. Example:

C

```
char grade = 'B';  
switch (grade) {  
    case 'A':  
        printf("Excellent!\n");  
        break;  
    case 'B':  
        printf("Good job!\n");  
        break;  
    case 'C':  
        printf("Average.\n");  
        break;  
    default:
```



```
printf("Needs improvement.\n");  
  
}
```

LAB EXERCISE:

Q1. Write a C program to check if a number is even or odd using an if-else statement.

Extend the program using a switch statement to display the month name based on the user's input (1 for January, 2 for February, etc.).

```
#include <stdio.h> int main() { int number; int  
monthNumber; // Check if a number is even  
or odd printf("Enter an integer: "); scanf("%d",  
&number); if (number % 2 == 0) { printf("%d is  
even.\n", number); } else { printf("%d is  
odd.\n", number); } // Display month name  
based on user input printf("\nEnter a month  
number (1-12): "); scanf("%d",  
&monthNumber); switch (monthNumber) {  
case 1: printf("January\n"); break; case 2:  
printf("February\n"); break; case 3:  
printf("March\n"); break; case 4:
```

```
printf("April\n"); break; case 5:  
printf("May\n"); break; case 6:  
printf("June\n"); break; case 7:  
printf("July\n"); break; case 8:  
printf("August\n"); break; case 9:  
printf("September\n"); break; case 10:  
printf("October\n"); break; case 11:  
printf("November\n"); break; case 12:  
printf("December\n"); break; default:  
printf("Invalid month number.\n"); } return 0;  
}
```

6. Looping in C

• THEORY EXERCISE:

Q1. Compare and contrast while loops, for loops, and do-while loops. Explain the scenarios in which each loop is most appropriate

Let's compare and contrast the three fundamental loop structures in C: while, for, and do-while.

1. while Loop:

- **Syntax:**

C

```
while (condition) {  
    // Code to be executed repeatedly  
}
```

- **Behavior:**

- **The while loop checks the condition *before* each iteration.**
- **If the condition is true, the loop body is executed.**
- **If the condition is false, the loop terminates.**
- **The loop body may not execute at all if the initial condition is false.**

- **Appropriate Scenarios:**

- **When you don't know in advance how many times the loop needs to iterate.**

- When the loop's continuation depends on a condition that might change during execution.¹

[1. For Loop vs While Loop: A Comprehensive Guide for Data Scientists | by Hey Amit | Medium](#)



medium.com

- Example: Reading data from a file until the end of the file is reached, or waiting for user input.

2. for Loop:

- Syntax:

C

```
for (initialization; condition;  
increment/decrement) {  
    // Code to be executed repeatedly  
}
```

. Behavior:

- **initialization: Executed once before the loop starts.**
- **condition: Checked before each iteration.**
- **increment/decrement: Executed after each iteration.**
- **The loop body is executed as long as the condition is true.**

. Appropriate Scenarios:

- **When you know in advance how many times the loop needs to iterate.**
- **When you need to iterate over a sequence of values (e.g., arrays).**

- **Example: Iterating through an array, or performing a specific task a fixed number of times.**

3. do-while Loop:

- **Syntax:**

C

```
do {  
    // Code to be executed repeatedly  
} while (condition);
```

- **Behavior:**
 - **The loop body is executed *at least once*, even if the initial condition is false.**
 - **The condition is checked *after* each iteration.**
 - **The loop continues as long as the condition is true.**
- **Appropriate Scenarios:**

- When you need to ensure that the loop body executes at least once.
- When you want to perform an action and then repeat it based on a condition.
- Example: Displaying a menu and getting user input until the user chooses to exit.

Comparison and Contrast:

Feature	while Loop	for Loop	do-while Loop
Condition Check	Before each iteration	Before each iteration	After each iteration
Minimum Iterations	0	0	1
Use Case	Unknown iterations	Known iterations	At least one iteration

	while	for (init;	do {}
Structure	(condition)	cond; incr)	while
	{}	{}	(cond);

Export to Sheets

Example Scenarios:

- **while Loop:**

C

```
int num = 1;

while (num <= 10) {
    printf("%d ", num);
    num++;
}
```

- **for Loop:**

C

```
for (int i = 0; i < 5; i++) {
    printf("Iteration %d\n", i);
}
```

- **do-while Loop:**

C

```
int choice;  
  
do {  
    printf("1. Do something\n2. Exit\nEnter  
choice: ");  
    scanf("%d", &choice);  
} while (choice != 2);
```

• **LAB EXERCISE:**

Q1. Write a C program to print numbers from 1 to 10 using all three types of loops (while, for, do-while).

```
#include <stdio.h> int main() { // Using while  
loop printf("Numbers from 1 to 10 using while  
loop:\n"); int i = 1; while (i <= 10) { printf("%d ",  
i); i++; } printf("\n"); // Using for loop  
printf("Numbers from 1 to 10 using for  
loop:\n"); for (int j = 1; j <= 10; j++) { printf("%d  
, j); } printf("\n"); // Using do-while loop  
printf("Numbers from 1 to 10 using do-while
```

```
loop:\n"); int k = 1; do { printf("%d ", k); k++; }  
while (k <= 10); printf("\n"); return 0; }
```

7. Loop Control Statements

• THEORY EXERCISE:

Q1.Explain the use of break, continue, and goto statements in C. Provide examples of each.

Let's explore the break, continue, and goto statements in C, which are used to control the flow of execution within loops and other control structures.

1. break Statement:

- . Purpose: The break statement is used to terminate the execution of a loop (for, while, do-while) or a switch statement prematurely.**
- . Behavior: When break is encountered, the program immediately exits the innermost loop or switch statement.**
- . Example (Loop):**

C

```
#include <stdio.h>
```

```
int main() {
```

```
    for (int i = 1; i <= 10; i++) {
```

```
        if (i == 5) {
```

```
            break; // Exit the loop when i is 5
```

```
        }
```

```
        printf("%d ", i);
```

```
    }
```

```
    printf("\nLoop terminated.\n");
```

```
    return 0;
```

```
}
```

. Example (switch):

C

```
#include <stdio.h>
```

```
int main() {
```

```
int choice = 2;  
switch (choice) {  
    case 1:  
        printf("Choice 1\n");  
        break;  
    case 2:  
        printf("Choice 2\n");  
        break;  
    case 3:  
        printf("Choice 3\n");  
        break;  
    default:  
        printf("Invalid choice\n");  
}  
return 0;  
}
```

2. continue Statement:

- **Purpose:** The continue statement is used to skip the remaining statements in the current iteration of a loop and proceed to the next iteration.¹

[1. www.geeksforgeeks.org](http://www.geeksforgeeks.org)

www.geeksforgeeks.org

- **Behavior:** When continue is encountered, the program jumps to the loop's update expression (in a for loop) or the loop's condition check (in a while or do-while loop).
- **Example:**

C

```
#include <stdio.h>
```

```
int main() {  
    for (int i = 1; i <= 10; i++) {  
        if (i % 2 == 0) {  
            continue; // Skip even numbers  
        }  
        printf("%d ", i);  
    }  
    printf("\n");  
    return 0;  
}
```

3. goto Statement:

- **Purpose:** The goto statement is used to transfer control to a labeled statement within the same function.
- **Behavior:** When goto is encountered, the program jumps to the specified label.
- **Caution:** goto statements are generally discouraged because they can make code difficult to read and understand

(spaghetti code). They can sometimes be useful in very specific error-handling scenarios, however.

. Example:

C

```
#include <stdio.h>
```

```
int main() {
```

```
    int num = 5;
```

```
    if (num < 10) {
```

```
        goto myLabel; // Jump to myLabel
```

```
    }
```

```
    printf("This line will not be printed.\n");
```

```
    myLabel:
```

```
    printf("Number is less than 10.\n");
```

```
    return 0;
```

```
}
```

• LAB EXERCISE:

Q1. Write a C program that uses the break statement to stop printing numbers when it reaches 5. Modify the program to skip printing the number 3 using the continue statement.

```
#include <stdio.h> int main() { // Using break  
statement printf("Printing numbers until 5  
using break:\n"); for (int i = 1; i <= 10; i++) { if (i  
== 5) { break; // Stop when i is 5 } printf("%d ",  
i); } printf("\n"); // Using continue statement  
printf("Printing numbers skipping 3 using  
continue:\n"); for (int j = 1; j <= 10; j++) { if (j ==  
3) { continue; // Skip printing 3 } printf("%d ",  
j); } printf("\n"); return 0; }
```

8. Functions in C

• THEORY EXERCISE:

Q1.What are functions in C? Explain function declaration, definition, and how to call a function. Provide examples.

Functions are fundamental building blocks in C programming. They allow you to

modularize your code, making it more organized, reusable, and easier to understand. Here's a breakdown of function declaration, definition, and calling, along with examples.

What are Functions in C?

A function is a block of code that performs a specific task. It can take input parameters (arguments), perform operations, and return a value. Functions help to:

- . Reduce code duplication: Reusable code blocks.**
- . Improve code organization: Break down complex tasks into smaller, manageable parts.**
- . Enhance readability: Make code easier to understand and maintain.**

1. Function Declaration (Prototype):

- . A function declaration, also called a prototype, informs the compiler about**

the function's name, return type, and parameters.

- . It's typically placed at the beginning of the program or in a header file.**
- . Syntax:**

C

```
return_type  
function_name(parameter_list);
```

- . Example:**

C

```
int add(int a, int b); // Function declaration  
  
void greet(char name[]); // Function  
declaration
```

2. Function Definition:

- . A function definition provides the actual code that the function executes.**
- . It includes the function header (same as the declaration) and the function body (the code within curly braces {}).**

. **Syntax:**

C

```
return_type function_name(parameter_list)
{
    // Function body (code)
    // ...
    return return_value; // If return_type is
not void
}
```

. **Example:**

C

```
int add(int a, int b) {
    int sum = a + b;
    return sum;
}

void greet(char name[]) {
    printf("Hello, %s!\n", name);
```

}

3. Calling a Function:

- . To use a function, you need to call it by its name and provide the necessary arguments.**
- . The function call executes the code within the function's definition.**
- . Example:**

C

```
#include <stdio.h>
```

```
// Function declaration
```

```
int add(int a, int b);
```

```
void greet(char name[]);
```

```
int main() {
```

```
    int result = add(5, 3); // Function call
```

```
    printf("Sum: %d\n", result);
```

```
greet("Alice"); // Function call
```

```
    return 0;  
}
```

```
// Function definitions
```

```
int add(int a, int b) {  
    int sum = a + b;  
    return sum;  
}
```

```
void greet(char name[]) {  
    printf("Hello, %s!\n", name);  
}
```

Explanation:

- In `main()`, `add(5, 3)` calls the `add` function with arguments 5 and 3. The return value (8) is stored in the `result` variable.
- `greet("Alice")` calls the `greet` function with the argument "Alice".
- The `add` function returns an integer value, while the `greet` function has a void return type (meaning it doesn't return a value).

• LAB EXERCISE:

Q1. Write a C program that calculates the factorial of a number using a function.

Include function declaration, definition, and call.

```
#include <stdio.h> // Function declaration
(long long factorial(int n); int
main() { int num; printf("Enter a non-negative
integer: "); scanf("%d", &num); if (num < 0) {
printf("Factorial is not defined for negative
numbers.\n"); } else { long long fact =
factorial(num); // Function call
printf("Factorial of %d is %lld\n", num, fact); }
```

```
return 0; } // Function definition long long  
factorial(int n) { if (n == 0) { return 1; } else {  
long long result = 1; for (int i = 1; i <= n; i++) {  
result *= i; } return result; } }
```

9. Arrays in C

• THEORY EXERCISE:

Q1.Explain the concept of arrays in C.

Differentiate between one-dimensional and multi-dimensional arrays with examples.

Arrays in C:

An array is a collection of elements of the same data type stored in contiguous memory locations. It provides a way to store multiple values of the same type under a single variable name.

Key Characteristics:

- Homogeneous Data: All elements in an array must be of the same data type.**
- Contiguous Memory: Array elements are stored in adjacent memory locations.**

- **Fixed Size:** The size of an array is fixed at compile time (or sometimes using variable length arrays in more recent C standards).
- **Zero-Based Indexing:** Array elements are accessed using indices starting from 0.

1. One-Dimensional Arrays:

- A one-dimensional array is like a linear list or a single row of elements.
- It has only one subscript (index) to access its elements.
- **Declaration:**

C

```
data_type array_name[size];
```

- **Example:**

C

```
#include <stdio.h>
```

```
int main() {
```



```
int numbers[5] = {10, 20, 30, 40, 50}; //  
Initialization
```

```
printf("Elements of the array:\n");  
for (int i = 0; i < 5; i++) {  
    printf("numbers[%d] = %d\n", i,  
numbers[i]);  
}  
return 0;  
}
```

2. Multi-Dimensional Arrays:

- A multi-dimensional array is an array of arrays. It allows you to store data in a tabular form (rows and columns).**
- It has multiple subscripts to access its elements.**
- Two-Dimensional Arrays:**
 - A two-dimensional array is like a table with rows and columns.**

- **Declaration:**

C

```
data_type array_name[rows][columns];
```

*** **Example:****

C

```
#include <stdio.h>
```

```
int main() {
```

```
    int matrix[3][3] = {
```

```
        {1, 2, 3},
```

```
        {4, 5, 6},
```

```
        {7, 8, 9}
```

```
    };
```

```
    printf("Elements of the 2D array:\n");
```

```
    for (int i = 0; i < 3; i++) { // Rows
```

```
        for (int j = 0; j < 3; j++) { // Columns
```

```

        printf("matrix[%d][%d] = %d\t", i, j,
matrix[i][j]);
    }
    printf("\n");
}
return 0;
}

```

• Higher-Dimensional Arrays:

- C supports arrays with more than two dimensions.
- For example, a three-dimensional array can be used to represent a cube.
- Declaration:

C

```
data_type array_name[dim1][dim2][dim3];
```

• LAB EXERCISE:

Q1. Write a C program that stores 5 integers in a one-dimensional array and prints them.

Extend this to handle a two-dimensional

array (3x3 matrix) and calculate the sum of all elements.

```
#include <stdio.h> int main() { // One-  
dimensional array int numbers[5] = {10, 20,  
30, 40, 50}; printf("One-dimensional array  
elements:\n"); for (int i = 0; i < 5; i++) {  
printf("%d ", numbers[i]); } printf("\n\n"); //  
Two-dimensional array (3x3 matrix) int  
matrix[3][3] = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };  
printf("Two-dimensional array (3x3  
matrix):\n"); for (int i = 0; i < 3; i++) { for (int j =  
0; j < 3; j++) { printf("%d ", matrix[i][j]); }  
printf("\n"); } // Calculate the sum of all  
elements in the matrix int sum = 0; for (int i =  
0; i < 3; i++) { for (int j = 0; j < 3; j++) { sum +=  
matrix[i][j]; } } printf("\nSum of all elements  
in the matrix: %d\n", sum); return 0; }
```

10. Pointers in C

• THEORY EXERCISE:

Q1.Explain what pointers are in C and how they are declared and initialized. Why are pointers important in C?

What are Pointers in C?

In C, a pointer is a variable that stores the memory address of another variable. Instead of holding a value directly, a pointer holds the location in memory where that value is stored.

Declaration:

- . To declare a pointer, you use the asterisk * symbol.**
- . The data type of the pointer must match the data type of the variable it points to.**
- . Syntax:**

C

data_type *pointer_name;

- . Example:**

C

int *ptr; // Pointer to an integer

char *charPtr; // Pointer to a character

float *floatPtr; // Pointer to a float

Initialization:

- . A pointer must be initialized before it is used.**
- . You can initialize a pointer with the address of another variable using the address-of operator &.**
- . You can also initialize a pointer to NULL (or 0) to indicate that it doesn't point to anything.**
- . Example:**

C

int num = 10;

**int *ptr = # // Initialize ptr with the
address of num**

char ch = 'A';

**char *charPtr = &ch; // Initialize charPtr
with the address of ch**

**int *nullPtr = NULL; // Initialize a null
pointer**

Accessing Values Through Pointers:

- . To access the value stored at the memory address pointed to by a pointer, you use the dereference operator *.**
- . Example:**

C

```
int num = 10;  
int *ptr = &num;
```

```
printf("Value of num: %d\n", num); //
```

Output: 10

```
printf("Value pointed to by ptr: %d\n", *ptr);  
// Output: 10
```

***ptr = 20; // Modify the value through the pointer**

printf("New value of num: %d\n", num); //

Output: 20

Importance of Pointers in C:

- 1. Dynamic Memory Allocation:**
 - **Pointers are essential for dynamic memory allocation using functions like malloc, calloc, and realloc.**
 - **They allow you to allocate memory during runtime, which is crucial for handling data structures of varying sizes.**
- 2. Passing Arguments by Reference:**
 - **Pointers enable you to pass arguments to functions by reference, allowing functions to modify the original variables.**

- **This is more efficient than passing by value, especially for large data structures.**

3. Working with Arrays:

- **Pointers and arrays are closely related in C.**
- **Array names can be treated as pointers to the first element of the array.**
- **Pointers can be used to traverse and manipulate array elements efficiently.**

4. Implementing Data Structures:

- **Pointers are fundamental for implementing complex data structures like linked lists, trees, and graphs.**
- **They allow you to create dynamic and flexible data structures.**

5. Efficient Memory Management:

- **Pointers provide low-level access to memory, enabling you to manage memory efficiently.**
- **This is crucial for system programming and embedded systems.**

6. Function Pointers:

- **C allows you to create pointers to functions, enabling you to pass functions as arguments to other functions and create callback mechanisms.**

• LAB EXERCISE:

Q1. Write a C program to demonstrate pointer usage. Use a pointer to modify the value of a variable and print the result.

```
#include <stdio.h> int main() { int num = 10;  
int *ptr; // Declare a pointer to an integer ptr  
= &num; // Initialize the pointer with the  
address of num printf("Original value of num:
```

```
%d\n", num); printf("Address of num: %p\n",  
(void *)ptr); // Print the memory address //  
Modify the value of num through the pointer  
*ptr = 20; printf("Modified value of num:  
%d\n", num); printf("Value pointed to by ptr:  
%d\n", *ptr); return 0; }
```

11. Strings in C

• THEORY EXERCISE:

Q1. Explain string handling functions like `strlen()`, `strcpy()`, `strcat()`, `strcmp()`, and `strchr()`. Provide examples of when these functions are useful.

Let's explore common string handling functions in C, explaining their purpose and providing usage examples.

String Handling Functions in C:

C provides a set of functions in the `string.h` header file for manipulating strings (character arrays).

1. `strlen()`:

- **Purpose:** Calculates the length of a string (excluding the null terminator \0).
- **Syntax:** `size_t strlen(const char *str);`
- **Example:**

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char text[] = "Hello, world!";
```

```
    size_t length = strlen(text);
```

```
    printf("Length of the string: %zu\n",  
length); // Output: 13
```

```
    return 0;
```

```
}
```

- **Usefulness:**
 - **Determining the size of a string before processing it.**

- **Allocating memory for dynamic string operations.**
- **Validating user input.**

2. strcpy():

- **Purpose: Copies a string from a source to a destination.**
- **Syntax: char *strcpy(char *dest, const char *src);**
- **Example:**

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char source[] = "Copy this!";
```

```
    char destination[20];
```

```
    strcpy(destination, source);
```

```
    printf("Copied string: %s\n", destination);  
// Output: Copy this!  
    return 0;  
}
```

- **Usefulness:**

- **Creating copies of strings.**
- **Assigning string literals to character arrays.**
- **Manipulating strings without modifying the original source.**
- **Caution: make sure the destination array is large enough to contain the source string.**

3. strcat():

- **Purpose: Concatenates (appends) one string to the end of another.**
- **Syntax: char *strcat(char *dest, const char *src);**
- **Example:**

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str1[50] = "Hello, ";
```

```
    char str2[] = "world!";
```

```
    strcat(str1, str2);
```

```
    printf("Concatenated string: %s\n", str1);
```

```
// Output: Hello, world!
```

```
    return 0;
```

```
}
```

. Usefulness:

- Building strings dynamically.
- Combining string fragments.
- Creating formatted output.

- **Caution:** Make sure the destination array is large enough to contain the concatenated string.

4. strcmp():

- **Purpose:** Compares two strings lexicographically (alphabetical order).
- **Syntax:** `int strcmp(const char *str1, const char *str2);`
- **Return Value:**
 - **0:** If the strings are equal.
 - **Negative value:** If str1 comes before str2 in lexicographical order.
 - **Positive value:** If str1 comes after str2 in lexicographical order.
- **Example:**

C

```
#include <stdio.h>
```

```
#include <string.h>
```



```
int main() {  
    char str1[] = "apple";  
    char str2[] = "banana";  
    int result = strcmp(str1, str2);  
    if (result == 0) {  
        printf("Strings are equal.\n");  
    } else if (result < 0) {  
        printf("str1 comes before str2.\n");  
    } else {  
        printf("str1 comes after str2.\n");  
    } // Output: str1 comes before str2.  
    return 0;  
}
```

. Usefulness:

- **Sorting strings.**
- **Searching for strings in a list.**
- **Comparing user input with stored values.**

5. strchr():

- **Purpose:** Finds the first occurrence of a character in a string.
- **Syntax:** `char *strchr(const char *str, int c);`
- **Return Value:**
 - A pointer to the first occurrence of the character.
 - NULL if the character is not found.
- **Example:**

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char text[] = "Hello, world!";
```

```
    char *ptr = strchr(text, 'w');
```

```
    if (ptr != NULL) {
```

```
printf("Character found at position:  
%ld\n", ptr - text); // Output: 7
```

```
} else {  
    printf("Character not found.\n");  
}  
return 0;  
}
```

- **Usefulness:**

- Searching for specific characters in a string.
- Parsing strings.
- Extracting substrings.

- **LAB EXERCISE:**

Q1. Write a C program that takes two strings from the user and concatenates them using strcat(). Display the concatenated string and its length using strlen()

```
#include <stdio.h> #include <string.h> int  
main() { char str1[100]; // Allocate enough
```

```
space for the concatenated string char
str2[50]; printf("Enter the first string: ");
fgets(str1, sizeof(str1), stdin); // Use fgets to
safely read strings printf("Enter the second
string: "); fgets(str2, sizeof(str2), stdin); // Use
fgets to safely read strings // Remove trailing
newline characters, if any str1[strcspn(str1,
"\n")] = 0; str2[strcspn(str2, "\n")] = 0; //
Concatenate the strings strcat(str1, str2); //
Display the concatenated string
printf("Concatenated string: %s\n", str1); //
Display the length of the concatenated string
printf("Length of concatenated string:
%zu\n", strlen(str1)); return 0; }
```

12. Structures in C

• THEORY EXERCISE:

Q1.Explain the concept of structures in C.

Describe how to declare, initialize, and access structure members.

Let's dive into the concept of structures in C, covering their declaration, initialization, and member access.

Structures in C:

A structure is a user-defined data type that allows you to group together variables of different data types under a single¹ name. It's similar to a record in other programming languages. Structures provide a way to create custom data types that represent real-world entities.

[1. www.prepbytes.com](http://www.prepbytes.com)

www.prepbytes.com

Declaration:

- To declare a structure, you use the **struct** keyword.
- The structure definition specifies the member variables and their data types.
- Syntax:

C

```
struct structure_name {  
    data_type member1;  
    data_type member2;  
    // ... more members  
}; // Don't forget the semicolon!
```

- Example:

C

```
struct Book {  
    char title[50];  
    char author[50];  
    int pages;  
    float price;
```

};

Initializing Structures:

- . You can initialize a structure variable at the time of declaration or later.**
- . Initialization at Declaration:**

C

```
struct Book book1 = {  
    "The C Programming Language",  
    "Brian Kernighan, Dennis Ritchie",  
    272,  
    29.99f  
};
```

- . Initialization Member by Member:**

C

```
struct Book book2;  
strcpy(book2.title, "Data Structures and  
Algorithms");
```

```
strcpy(book2.author, "Alfred V. Aho, John E.  
Hopcroft, Jeffrey D. Ullman");
```

```
book2.pages = 400;
```

```
book2.price = 39.95f;
```

Accessing Structure Members:

- . To access the members of a structure, you use the dot operator (.).**
- . Syntax:**

C

```
structure_variable.member_name;
```

- . Example:**

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Book {
```

```
    char title[50];
```

```
    char author[50];
```



```
int pages;  
float price;  
};
```

```
int main() {  
    struct Book book1 = {  
        "The C Programming Language",  
        "Brian Kernighan, Dennis Ritchie",  
        272,  
        29.99f  
    };  
  
    printf("Title: %s\n", book1.title);  
    printf("Author: %s\n", book1.author);  
    printf("Pages: %d\n", book1.pages);  
    printf("Price: %.2f\n", book1.price);  
  
    return 0;
```

}

Pointers to Structures:

- . You can also use pointers to structures.**
- . To access members of a structure through a pointer, you use the arrow operator (->).**
- . Syntax:**

C

structure_pointer->member_name;

- . Example:**

C

#include <stdio.h>

#include <string.h>

struct Book {

char title[50];

char author[50];

int pages;

```
float price;

};

int main() {

    struct Book book1 = {

        "The C Programming Language",
        "Brian Kernighan, Dennis Ritchie",
        272,
        29.99f

    };

    struct Book *ptr = &book1;

    printf("Title: %s\n", ptr->title);
    printf("Author: %s\n", ptr->author);
    printf("Pages: %d\n", ptr->pages);
    printf("Price: %.2f\n", ptr->price);
```

```
    return 0;  
}
```

Key Points:

- **Structures help in creating complex data types.**
 - **The members of a structure can be of different data types.**
 - **The dot operator (.) is used to access structure members through a structure variable.**
 - **The arrow operator (->) is used to access structure members through a structure pointer.**
 - **Structures can be nested (a structure can contain other structures).**
 - **Structures can be used as function parameters and return types.**
- **LAB EXERCISE:**

Q1. Write a C program that defines a structure to store a student's details (name, roll number, and marks). Use an array of structures to store details of 3 students and print them.

```
#include <stdio.h> #include <string.h> struct  
Student { char name[50]; int rollNumber;  
float marks; }; int main() { struct Student  
students[3]; // Array of 3 Student structures  
// Input student details for (int i = 0; i < 3; i++) {  
printf("Enter details for student %d:\n", i + 1);  
printf("Name: "); fgets(students[i].name,  
sizeof(students[i].name), stdin);  
students[i].name[strcspn(students[i].name,  
"\n")] = 0; // Remove trailing newline  
printf("Roll Number: "); scanf("%d",  
&students[i].rollNumber); getchar(); //  
Consume the newline character left by scanf  
printf("Marks: "); scanf("%f",  
&students[i].marks); getchar(); // Consume  
the newline character left by scanf } // Print  
student details printf("\nStudent
```

```
Details:\n"); for (int i = 0; i < 3; i++) {  
printf("Student %d:\n", i + 1); printf("Name:  
%s\n", students[i].name); printf("Roll  
Number: %d\n", students[i].rollNumber);  
printf("Marks: %.2f\n", students[i].marks);  
printf("\n"); } return 0; }
```

13. File Handling in C

• THEORY EXERCISE:

Q1.Explain the importance of file handling in C. Discuss how to perform file operations like opening, closing, reading, and writing files.

File handling in C is crucial for persistent data storage and retrieval. It allows programs to interact with external files, enabling them to read data from files, write data to files, and manage data beyond the program's execution lifespan.

Importance of File Handling:

1. Data Persistence:

- **Files provide a way to store data permanently, even after the program terminates.**
- **This is essential for applications that need to save user data, configuration settings, or large datasets.**

2. Data Sharing and Exchange:

- **Files facilitate data sharing between different programs and users.**
- **Data can be exported to files and imported by other applications.**

3. Data Organization and Management:

- **Files allow you to organize and manage large amounts of data in a structured manner.**
- **You can use files to store data in various formats, such as text, binary, or structured data.**

4. Configuration and Logging:

- **Files are used to store program configuration settings and log program events.**
- **This helps in customizing program behavior and debugging.**

File Operations in C:

1. Opening a File:

- **The fopen() function is used to open a file.**
- **Syntax: FILE *fopen(const char *filename, const char *mode);**
- **filename: The name of the file to open.**
- **mode: The mode in which to open the file (e.g., "r" for reading, "w" for writing, "a" for appending).**
- **Return value: A file pointer (FILE *) if successful, or NULL if an error occurs.**
- **Example:**

C

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *filePtr;
```

```
    filePtr = fopen("myfile.txt", "w"); // Open  
for writing
```

```
    if (filePtr == NULL) {
```

```
        printf("Error opening file.\n");
```

```
        return 1;
```

```
    }
```

```
    // ... file operations ...
```

```
    fclose(filePtr);
```

```
    return 0;
```

```
}
```

2. Closing a File:

- The **fclose()** function is used to close a file.
- **Syntax: int fclose(FILE *filePtr);**
- **filePtr:** The file pointer returned by **fopen()**.
- **Return value: 0 if successful, or EOF if an error occurs.**
- **It's crucial to close files after use to release system resources and ensure data integrity.**

3. Reading from a File:

- **Character-by-Character:**
 - **int fgetc(FILE *filePtr);:** Reads a single character from the file.
 - **Returns the character as an int or EOF on error or end of file.**
- **Line-by-Line:**

- **char *fgets(char *str, int n, FILE *filePtr);**: Reads a line of text from the file into the string str.
- Reads up to n - 1 characters or until a newline or EOF is encountered.
- **Formatted Input:**
 - **int fscanf(FILE *filePtr, const char *format, ...);**: Reads formatted input from the file, similar to scanf().
- **Example:**

C

```
#include <stdio.h>
```

```
int main() {
```

```
FILE *filePtr = fopen("myfile.txt", "r");
```

```
if (filePtr == NULL) {
```

```
printf("Error opening file.\n");
```

```
        return 1;
    }

    char buffer[100];

    while (fgets(buffer, sizeof(buffer), filePtr)
!= NULL) {
        printf("%s", buffer);
    }

    fclose(filePtr);

    return 0;
}
```

4. Writing to a File:

- **Character-by-Character:**

- **int fputc(int c, FILE *filePtr);:**

- Writes a single character c to the file.**

- **Line-by-Line:**

- **int fputs(const char *str, FILE *filePtr);** Writes the string str to the file.
- **Formatted Output:**
 - **int fprintf(FILE *filePtr, const char *format, ...);** Writes formatted output to the file, similar to printf().
- **Example:**

C

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *filePtr = fopen("myfile.txt", "w");
```

```
    if (filePtr == NULL) {
```

```
        printf("Error opening file.\n");
```

```
        return 1;
```

```
    }
```

```
fprintf(filePtr, "Hello, file!\n");  
fputs("This is a line of text.\n", filePtr);  
  
fclose(filePtr);  
  
return 0;  
  
}
```

File Modes:

- "r": Read mode (file must exist).**
- "w": Write mode (creates a new file or overwrites an existing one).**
- "a": Append mode (writes to the end of an existing file or creates a new one).**
- "r+": Read and write mode (file must exist).**
- "w+": Read and write mode (creates a new file or overwrites an existing one).**
- "a+": Read and append mode (writes to the end of an existing file or creates a new one).**

- "rb", "wb", "ab", "r+b", "w+b", "a+b": Binary modes (for binary files).

- **LAB EXERCISE:**

Q1. Write a C program to create a file, write a string into it, close the file, then open the file again to read and display its contents.

```
#include <stdio.h> #include <stdlib.h> // For
exit() int main() { FILE *filePtr; char
writeString[] = "This is a string written to the
file.\n"; char readString[100]; // Buffer to
read from the file // Write to the file filePtr =
fopen("myfile.txt", "w"); // Open in write
mode if (filePtr == NULL) { printf("Error
creating/opening file for writing.\n"); return 1;
// Indicate an error } fprintf(filePtr, "%s",
writeString); // Write the string to the file
fclose(filePtr); // Close the file // Read from
the file filePtr = fopen("myfile.txt", "r"); //
Open in read mode if (filePtr == NULL) {
printf("Error opening file for reading.\n");
return 1; // Indicate an error }
```

```
printf("Contents of the file:\n"); while  
(fgets(readString, sizeof(readString), filePtr)  
!= NULL) { printf("%s", readString); // Print the  
read string } fclose(filePtr); // Close the file  
return 0; }
```